

<https://github.com/ingesoft5/Taller1-CICD-g5-A00378149>

Reflexión:

1. Ruptura de Silos Dev-Ops

En el modelo tradicional, Dev escribe código y Ops lo despliega, pero nadie entiende el trabajo del otro. Esto causa dos problemas: primero, entregas lentas porque hay que pasar el código por correo y esperar a que Ops lo configure; segundo, errores en producción porque los desarrolladores no conocen el entorno real y las pruebas locales no reflejan la realidad.

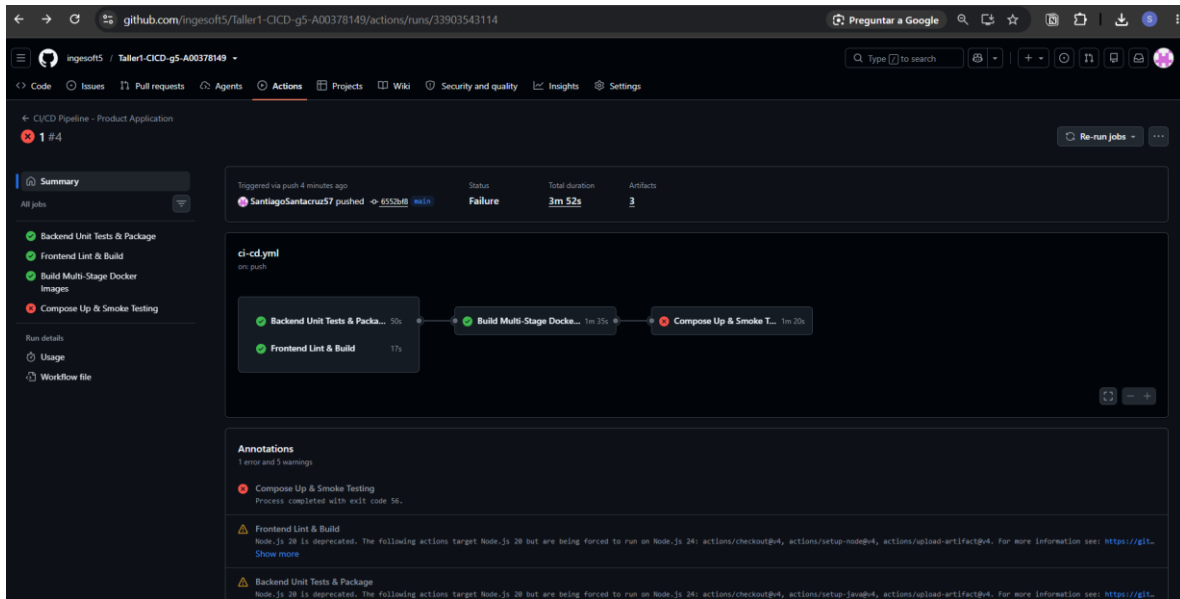
El pipeline con jobs interdependientes (test-backend, test-frontend, build-docker, smoke-test) rompe esto porque todos ven el mismo flujo. Las pruebas unitarias ya no son solo de Dev, los Dockerfiles obligan a pensar en producción desde el inicio, y el smoke-test replica el entorno real con contenedores. Si algo falla, lo ven todos, no hay "culpa" de un solo equipo.

2. Principios CALMS

Cultura: El pipeline automático en cada push fomenta que todos sean responsables del código, no solo de "terminar mi parte".

Automatización: Todo está automatizado: pruebas con Maven, lint con npm, construcción de imágenes con Docker, y despliegue con compose. No hay pasos manuales que puedan fallar por error humano.

Lean (Eficiencia): Usamos caching en Maven y Docker para no repetir trabajo innecesario, y las imágenes multi-stage con alpine reducen el tamaño final, aplicando la filosofía de "solo lo necesario".



Conversación con la IA:

Esto que pegaste es el **Dockerfile** del backend (formato correcto, sin problema — de hecho se ve bien), pero el error que estás viendo (go-yaml load error) viene específicamente del parser de **YAML**, y un Dockerfile no es YAML — es su propia sintaxis. Por eso este archivo no puede ser la causa del error.

El error está en tu **docker-compose.yml**, que es un archivo distinto. Necesito ver ese archivo, no el Dockerfile.

¿Puedes pegar el contenido completo de tu docker-compose.yml? O mejor, corre esto y pégame la salida:

```
bash
```

```
cat -A docker-compose.yml | sed -n '1,10p'
```

Eso me va a mostrar exactamente qué carácter raro hay en la línea 8 (tabs, comillas curvas, BOM, etc.) para poder darte el fix exacto en vez de seguir adivinando.

Hay varias capas de validación, de más rápida a más completa. Te recomiendo pasarlas todas antes del push final.

1. Validación básica de sintaxis YAML (la más rápida)

bash

```
python3 -c "import yaml; yaml.safe_load(open('.github/workflows/ci-cd.yml'))" &&  
echo "YAML válido 
```

Si hay un error de indentación, tabs, o comillas raras (como el que tuviste antes con el docker-compose.yml), esto te lo va a mostrar con línea y columna exactas.

2. actionlint — linter específico para GitHub Actions (la más importante)

Esta es la herramienta correcta porque no solo valida YAML genérico, sino la sintaxis específica de Actions: nombres de jobs, needs que referencian jobs inexistentes, expresiones `${{ }}` mal formadas, versiones de actions, etc.

bash

Instalar una vez

```
curl -s https://raw.githubusercontent.com/rhysd/actionlint/main/scripts/download-  
actionlint.bash | bash
```

Correr sobre tu workflow

```
./actionlint .github/workflows/ci-cd.yml
```

Esto te va a detectar cosas como:

- Si needs: [test-backend, test-frontend] referencia un nombre de job que no existe o está mal escrito.
- Si usas una versión de action que no existe (actions/checkout@v99).
- Errores de tipo en expresiones (`${{ github.sha }}` mal usado).
- Steps sin uses ni run.

3. docker compose config para la parte de Compose

Si tu smoke-test job depende de un docker-compose.yml correcto, válidalo aparte:

bash

```
docker compose config > /dev/null && echo "docker-compose OK"
```

4. Simular el workflow localmente con act (la más completa, opcional)

act ejecuta tus GitHub Actions **localmente en Docker**, simulando el runner real, sin necesidad de hacer push. Es lo más cercano a "probarlo de verdad" sin gastar corridas en GitHub:

```
bash
```

```
# Instalar (Linux)
```

```
curl -s https://raw.githubusercontent.com/nektos/act/master/install.sh | sudo bash
```

```
# Correr un job específico
```

```
act -j test-backend
```

```
act -j smoke-test
```

Ojo: act no soporta 100% de las features de Actions (por ejemplo, self-hosted runners los trata distinto, y algunas actions oficiales pueden comportarse diferente). Es útil para detectar errores obvios rápido, pero no reemplaza la corrida real en GitHub para el smoke-test con Docker Compose.

5. GitHub CLI — verificar el estado de una corrida ya hecha

Si ya hiciste push y quieres revisar resultados sin entrar al navegador:

```
bash
```

```
gh run list --workflow=ci-cd.yml
```

```
gh run view --log
```

(Requiere gh auth login una vez.)

Mi recomendación de flujo antes de cada push:

```
bash
```

```
python3 -c "import yaml; yaml.safe_load(open('.github/workflows/ci-cd.yml'))" && \
```

```
./actionlint .github/workflows/ci-cd.yml && \
```

```
docker compose config > /dev/null && \
```

```
echo "Todo validado, listo para push "
```

Si actionlint te marca algo, pégame la salida y lo resolvemos antes de gastar una corrida real en GitHub Actions.

i keep getting this error but im an admin in the repo Username for '<https://github.com>': SantiagoSantacruz57 Password for '<https://SantiagoSantacruz57@github.com>': remote: Permission to ingesoft5/Taller1-CICD-g5-A00378149.git denied to SantiagoSantacruz57. fatal: unable to access '<https://github.com/ingesoft5/Taller1-CICD-g5-A00378149.git/>': The requested URL returned error: 403

Esto no es un problema de permisos de admin — es que **GitHub ya no acepta tu contraseña de cuenta** para operaciones de git por HTTPS desde agosto de 2021. Necesitas un **Personal Access Token (PAT)** en vez de tu password, incluso siendo admin del repo.

Solución rápida: usar un Personal Access Token

1. Genera un token en GitHub:

- Ve a <https://github.com/settings/tokens>
- Clic en "**Generate new token**" → "**Generate new token (classic)**"
- Dale un nombre descriptivo (ej. "taller-cicd-local")
- Marca el scope **repo** (acceso completo a repos privados/públicos)
- Genera y **copia el token inmediatamente** — no lo vuelves a ver después

2. Úsalo en vez de tu password:

Cuando git te pida usuario y contraseña de nuevo:

Username: SantiagoSantacruz57

Password: [pega aquí el token, NO tu contraseña de GitHub]

Mejor opción: evita que te lo pida cada vez

Opción A — Cachear credenciales:

```
bash
```

```
git config --global credential.helper cache
```

Guarda el token en memoria por un tiempo limitado.

Opción B — Guardar credenciales en disco (más cómodo, menos seguro):

```
bash
```

```
git config --global credential.helper store
```

La primera vez que uses el token, queda guardado en ~/.git-credentials y no te lo vuelve a pedir.

Opción recomendada: cambiar a SSH (evita todo esto)

Si vas a trabajar seguido con este repo, SSH es más estable a largo plazo que HTTPS + token:

```
bash
```

```
# Generar llave SSH (si no tienes una)
```

```
ssh-keygen -t ed25519 -C "tu-email@ejemplo.com"
```

```
# Copiar la llave pública
```

```
cat ~/.ssh/id_ed25519.pub
```

Luego pégala en <https://github.com/settings/keys> → "New SSH key".

Y cambia el remote del repo de HTTPS a SSH:

```
bash
```

```
git remote set-url origin git@github.com:ingesoft5/Taller1-CICD-g5-A00378149.git
```

Después de eso, git push ya no te pide usuario/contraseña nunca más.

Para confirmar que ya quedó bien, intenta:

```
bash
```

```
git push origin main
```

Si usaste el token, debería pasar sin el error 403. Si sigue fallando, verifica que el token tenga el scope repo marcado y que no haya expirado.